

Lecture 10

Cluster Analysis

10.1

Contents

1	Introduction	1
2	Dissimilarity and Similarity Measures	1
3	Agglomerative Algorithms in Cluster Analysis	3
3.1	Hierarchical Methods	3
3.2	Non-Hierarchical Methods (<i>K</i> -means)	7
4	Examples	9
4.1	Classification of US Public Utilities (Both Variables and Cases)	9
4.2	Classification of Languages	14
4.3	Classification of Scotch Whiskies	18

See Johnson and Wichern 2007, Applied Multivariate Statistical Analysis, Pearson Education.

10.2

1 Introduction

Goal

Find possible groupings of subjects (or variables) such that subjects (or variables) within groups are as more similar as possible and subjects (or variables) belonging to different groups are as different as possible.

Cluster analysis is an *exploratory method*.

The closeness measure between two objects, y_r and y_s (belonging to the p -dimensional space) is said a proximity measure. We can have:

- dissimilarity measures,
- similarity measures.

10.3

2 Dissimilarity and Similarity Measures

Dissimilarity Measure

Let $\mathbf{y}_r$ and $\mathbf{y}_s$ be two objects in the p -dimensional space.

A dissimilarity measure satisfies the following properties:

1. $d_{rs} \geq 0$
2. $d_{rs} = 0$ if $\mathbf{y}_r = \mathbf{y}_s$

3. $d_{rs} = d_{sr}$ (symmetric property)
4. The dissimilarity measure is said to be a *metric* when, in presence of a generic object $\mathbf{y}_q$ in the p -dimensional space, it holds

$$d_{rs} \leq d_{rq} + d_{qs} \text{ (triangular inequality).}$$

A dissimilarity measure decreases when objects are closer.

10.4

Examples of Dissimilarity Measures

- The *square* of the Euclidean distance

$$d_{rs}^2 = \|\mathbf{y}_r - \mathbf{y}_s\|^2.$$

- The *square* of the weighted Euclidean distance

$$d_{rs}^2 = (\mathbf{y}_r - \mathbf{y}_s)' [\text{diag}(\boldsymbol{\Sigma})]^{-1} (\mathbf{y}_r - \mathbf{y}_s).$$

- The *square* of the Mahalanobis distance

$$d_{rs}^2 = (\mathbf{y}_r - \mathbf{y}_s)' \boldsymbol{\Sigma}^{-1} (\mathbf{y}_r - \mathbf{y}_s).$$

- The Minkowski distance

$${}_M d_{rs} = \left(\sum_{j=1}^p |y_{rj} - y_{sj}|^\lambda \right)^{\frac{1}{\lambda}}$$

- if $\lambda = 1$ we have the ℓ_1 (city block or Manhattan) distance.
- if $\lambda = 2$ we have the ℓ_2 Euclidean distance.
- if $\lambda \rightarrow \infty$ we have the Lagrange distance

$$d_{rs} = \max_j |y_{rj} - y_{sj}|.$$

Other particular distances exist.

We remind the Canberra distance

$$d_{rs} = \sum_{j=1}^p \frac{|y_{rj} - y_{sj}|}{(y_{rj} + y_{sj})}$$

that is used with asymmetric distributions or in presence of anomalous data.

10.5

10.6

Similarity Measure

Let $\mathbf{y}_r$ and $\mathbf{y}_s$ be two objects in the p -dimensional space.

A similarity measure satisfies the following properties:

1. $0 \leq s_{rs} \leq 1$
2. $s_{rs} = 1$ if $\mathbf{y}_r = \mathbf{y}_s$
3. $s_{rs} = s_{sr}$ (symmetric property)

A similarity measure increases when objects are closer.

An example of similarity measure is the Pearson's correlation distance.

10.7

3 Agglomerative Algorithms in Cluster Analysis

Two kind of aggregation methods are available

- Hierarchical Agglomerative
- Non-Hierarchical Agglomerative

10.8

Hierarchical Agglomerative

This is a 'bottom up' approach.

- Each observation starts in its own cluster.
- The most similar objects are first grouped, and these initial groups are merged according to their similarities. Eventually, as similarity decreases all subgroups are merged into a single cluster.
- If a subject is assigned to a group according to a certain criterion, it *cannot be moved to another group*.

10.9

Non-Hierarchical Agglomerative

- The number of groups is defined 'a priori'.
- Objects are assigned to groups until the criterion used to define the aggregation is optimized.
- The algorithm rule allows that a subject assigned to a group at a certain step can be moved to another group at a subsequent step.

10.10

3.1 Hierarchical Methods

Let $\mathbf{D}$ be a $n \times n$ dissimilarity matrix, whose generic element d_{ij} is the dissimilarity between objects i and j .

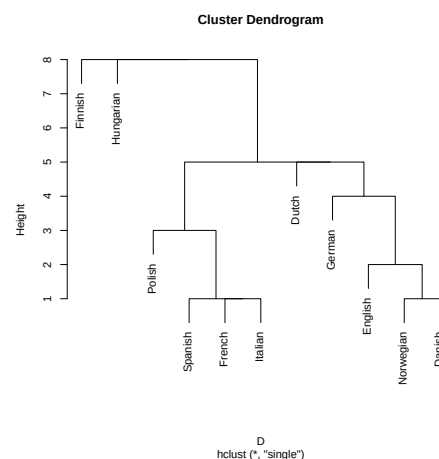
$\mathbf{D}$ resumes dissimilarities among all n initial objects (clusters).

1. Search $\mathbf{D}$ for the nearest pair R, S of most similar clusters and merge R and S , by labeling the new cluster as $\{RS\}$.
2. Update the entries in the dissimilarity matrix $\mathbf{D}$ by (a) deleting rows and columns corresponding to clusters R and S and (b) adding a row and column giving distances between $\{RS\}$ and all the remaining objects calculated with *an appropriate criterion*.
3. Repeat $(n - 1)$ times steps 1 and 2 until all n initial clusters are collapsed in a unique cluster.

Record the identity of clusters that are merged and the levels (distances or similarities) at which the mergers take place.

The final aggregation scheme can be represented with a dendrogram.

10.11



10.12

A dendrogram is a graph that illustrates the mergers made at successive levels. It has the following properties.

- It is a tree chart, vertical or horizontal, that represents successive partitions.
- The *leaves* of the tree are the initial objects.
- The graph shows distances at which clusters (objects) are merged.
The height of each branch in the plot is proportional to the value of the intergroup dissimilarity between its two parents/daughters (the leaves representing individual observations are all plotted at zero height).
- The dendrogram shows partitions that can be obtained at increasing distance levels.
- A 'cut' might be suggested where there is the longest branch (in order to find an optimum number of groups).
- More reasonable cut points might exist.
- The last 'leaves' that merge to the unique final cluster might be outlier objects.

10.13

Observe that there is no definitive answer on where to cut the dendrogram, since cluster analysis is essentially an exploratory approach; the interpretation of the resulting hierarchical structure is context-dependent and often several solutions are equally good from a theoretical point of view.

10.14

We have to define what does *an appropriate criterion* at step 2 of the hierarchical methods algorithm mean.

Usually one of the following criteria is used.

- Single linkage (Nearest-Neighbor)
- Complete linkage (Farthest-Neighbor)
- Average linkage
- Ward (incremental sum of squares)

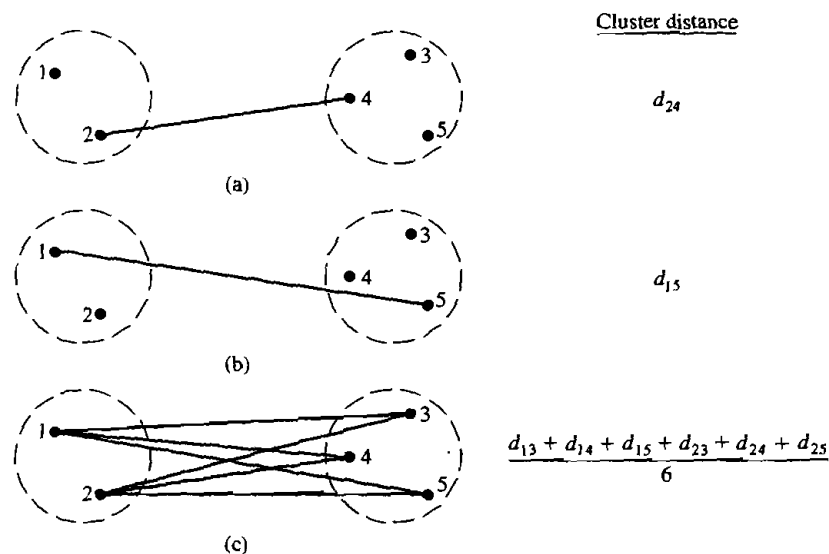


Figure 12.2 Intercluster distance (dissimilarity) for (a) single linkage, (b) complete linkage, and (c) average linkage.

Figure extracted from Johnson and Wichern 2007, Applied Multivariate Statistical Analysis.

10.15

Single linkage (Nearest-Neighbor)

The distance between a group $\{RS\}$ and an object (or group) Q is defined as

$$d_{\{RS\},Q} = \min(d_{RQ}, d_{SQ}).$$

For example let

$$\mathbf{D} = \begin{array}{c|ccc} & a & b & c & d \\ \hline a & 0 & & & \\ b & 1 & 0 & & \\ c & 3 & 2 & 0 & \\ d & 7 & 6 & 4 & 0 \end{array}$$

The first group will be $\{ab\}$

The criterion $d_{\{RS\},Q} = \min(d_{RQ}, d_{SQ})$ updates matrix $\mathbf{D}$ as

$$\mathbf{D}_1 = \begin{array}{c|ccc} & ab & c & d \\ \hline ab & 0 & & \\ c & 2 & 0 & \\ d & 6 & 4 & 0 \end{array}$$

The second group will be $\{abc\}$

The single linkage criterion updates $\mathbf{D}_1$ as

$$\mathbf{D}_2 = \begin{array}{c|cc} & abc & d \\ \hline abc & 0 & \\ d & 4 & 0 \end{array}$$

The final group is $\{abcd\}$.

Complete linkage (Farthest-Neighbor)

The distance between a group $\{RS\}$ and an object (or group) Q is defined as

$$d_{\{RS\},Q} = \max(d_{RQ}, d_{SQ}).$$

For example let

$$\mathbf{D} = \begin{array}{c|ccc} & a & b & c & d \\ \hline a & 0 & & & \\ b & 1 & 0 & & \\ c & 3 & 2 & 0 & \\ d & 7 & 6 & 4 & 0 \end{array}$$

The first group will be $\{ab\}$

The criterion $d_{\{RS\},Q} = \max(d_{RQ}, d_{SQ})$ updates matrix $\mathbf{D}$ as

$$\mathbf{D}_1 = \begin{array}{c|ccc} & ab & c & d \\ \hline ab & 0 & & \\ c & 3 & 0 & \\ d & 7 & 4 & 0 \end{array}$$

The second group will be $\{abc\}$

The complete linkage criterion updates $\mathbf{D}_1$ as

$$\mathbf{D}_2 = \begin{array}{c|cc} & abc & d \\ \hline abc & 0 & \\ d & 7 & 0 \end{array}$$

The final group is $\{abcd\}$.

Average linkage

Average linkage treats the distance between two clusters as the average distance between all pairs of items where one member of a pair belongs to each cluster.

Therefore, the distance between a cluster $\{RS\}$ and an object (or group) Q is defined as

$$d_{\{RS\},Q} = \frac{\sum_i \sum_k d_{ik}}{n_{\{RS\}} n_Q}$$

where d_{ik} is the distance between object i in cluster $\{RS\}$ and object k in the cluster Q , and $n_{\{RS\}}$ and n_Q are the number of items in clusters $\{RS\}$ and Q , respectively.

10.20

For example let

$$\mathbf{D} = \begin{array}{c|ccc} & a & b & c & d \\ \hline a & 0 & & & \\ b & 1 & 0 & & \\ c & 3 & 2 & 0 & \\ d & 7 & 6 & 4 & 0 \end{array}$$

The first cluster will be $\{ab\}$

The criterion $d_{\{RS\},Q} = \frac{\sum_i \sum_k d_{ik}}{n_{\{RS\}} n_Q}$ updates matrix $\mathbf{D}$ as

$$\mathbf{D}_1 = \begin{array}{c|cc} & ab & c & d \\ \hline ab & 0 & & \\ c & (3+2)/2 & 0 & \\ d & (7+6)/2 & 4 & 0 \end{array}$$

The second cluster will be $\{abc\}$

The average linkage criterion updates $\mathbf{D}_1$ as

$$\mathbf{D}_2 = \begin{array}{c|c} & abc & d \\ \hline abc & 0 & \\ d & (7+6+4)/3 & 0 \end{array}$$

The final cluster is $\{abcd\}$.

10.21

Ward (incremental sum of squares)

Ward considered hierarchical clustering procedures based on minimizing the 'loss of information' from joining two groups.

The method is usually implemented with loss of information taken to be as an increase in an error sum of squares ESS.

10.22

For a given cluster k , let ESS_k be the sum of the squared deviations of every object in the cluster from the cluster mean (centroid).

If there are K cluster let $ESS = \sum_{k=1}^K ESS_k$.

At each step in the analysis, the union of every possible pair of clusters is considered, and the two clusters whose combination results in the smallest increase in ESS (minimum loss of information) are joined.

ESS ranges between $\min ESS = 0$ when all clusters consist of single objects and $\max ESS = \sum_{i=1}^n (y_i - \bar{y})'(y_i - \bar{y})$ when all objects are combined in a single group.

10.23

Which criterion?

There is no unique answer, but the choice can be made according to the so-called minimal well-structured partition.

Minimal well-structured Partition

An agglomerative method should have some properties:

- The maximum distance within clusters has to be lower than the minimum distance between clusters, and the number of clusters that ensures this property must be as small as possible.
- The partition must be invariant to monotone transformations of distances (this holds for the single and the complete linkages)

10.24

Complete linkage is commonly used. Its drawback is its ability in finding elliptical groups. The single linkage can find also non elliptical structures but can be confused in presence of near overlapping of elliptical structures, and has the so-called chaining effect drawback; it can merge very far objects if these are joined by a sequence of intermediate objects.

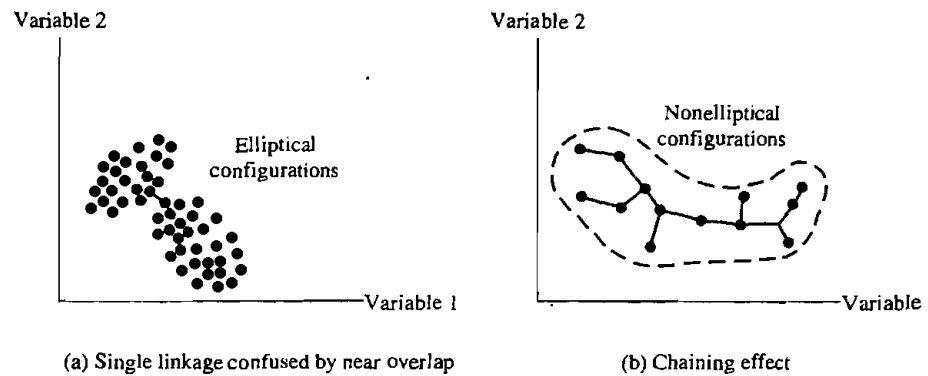


Figure 12.5 Single linkage clusters.

Figure extracted from Johnson and Wichern 2007, Applied Multivariate Statistical Analysis.

10.25

3.2 Non-Hierarchical Methods (K -means)

In its simplest version, the classification process is composed of these three steps.

1. K initial points, named centroids, are first selected. K values in $\mathfrak{R}^p$, the p -dimensional euclidean space.
2. Proceed through the list of items, assigning an item to the cluster whose centroid (mean) is nearest. (Distance is usually computed using Euclidean distance with either standardized or unstandardized observations.)
Recalculate the centroid for the cluster receiving the new item and for the cluster losing the item.
3. Repeat step 2 until no more reassignments take place.

10.26

For example let consider the following data

	X_1	X_2
a	5	3
b	-1	1
c	1	-2
d	-3	-2

Let the initial centroid be given by the averages of groups $\{ab\}$ and $\{cd\}$ (this assumes that we have already decided that the final solution is made up by 2 groups); the initial solution is characterized by centroids

	<i>centroids</i>	
<i>cluster</i>	$\bar{x}_1$	$\bar{x}_2$
ab	2	2
cd	-1	-2

Let us compute the square Euclidean distances of each initial object (a, b, c and d) from the group centroids. We have

$$d_{\{ab\},a}^2 = (5 - 2)^2 + (3 - 2)^2 = 10$$

$$d_{\{cd\},a}^2 = (5 + 1)^2 + (3 + 2)^2 = 61$$

Has a to be reassigned?

Since a is closer to $\{ab\}$ than to $\{cd\}$ it is not reassigned.

$$d_{\{ab\},b}^2 = (-1 - 2)^2 + (1 - 2)^2 = 10$$

$$d_{\{cd\},b}^2 = (-1 + 1)^2 + (1 + 2)^2 = 9$$

Has b to be reassigned?

Yes, b is reassigned to cluster $\{cd\}$, giving cluster $\{bcd\}$ and the following updated coordinates of the centroids

	<i>centroids</i>	
<i>cluster</i>	$\bar{x}_1$	$\bar{x}_2$
a	5	3
bcd	-1	-1

Again each item is checked for reassignment. Computing the squared Euclidean distances gives the following

	<i>objects</i>			
<i>cluster</i>	a	b	c	d
a	0	40	41	89
bcd	52	4	5	5

No object needs to be reassigned and the algorithm stops.

To check the stability of the clustering, it is desirable to rerun the algorithm with a new initial partition.

Advantages and disadvantages

- It can be used to group very big data sets ($n \simeq 1,000,000$).
- It does not show how objects are successively aggregated.
- It is a fast method (has low computation times).
- It can be repeated with different number of groups.
- Its use is suggested when one is more interested in creating *types of objects* than in aggregating single objects.

In presence of big data sets guesses of the number of groups and initial centroids can be obtained by applying hierarchical cluster analysis to a small sample of observations representative of all data.

Namely, there are strong arguments for not fixing the number of clusters, K , in advance, including the following:

- If two or more seed points (initial clusters) inadvertently lie within a single cluster, their resulting clusters will be poorly differentiated.
- The existence of an outlier might produce at least one group with very disperse items.
- Even if the population is known to consist of K groups, the sampling method may be such that data from the rarest group do not appear in the sample. Forcing the data into K groups would lead to nonsensical clusters.

4 Examples

We apply clustering to the following problems

- classification of US public utilities (both variables and cases)
- classification of languages
- classification of Scotch whiskies

10.32

4.1 Classification of US Public Utilities (Both Variables and Cases)

The data set `publiccompanies.dat` contains information about 22 US public utilities

- X_1 : fixed-charge coverage ratio (income/debt)
- X_2 : rate of return on capital
- X_3 : cost of KW capacity in place
- X_4 : annual load factor
- X_5 : peak kWh demand growth from 1974 to 1975
- X_6 : sales (kWh use per year)
- X_7 : percent nuclear
- X_8 : total fuel costs (cents per kWh)

10.33

```
pubcomp <- read.table("publiccompanies.dat")
head(pubcomp)
##      V1    V2    V3    V4    V5    V6    V7    V8    V9
## 1 1.06   9.2  151  54.4   1.6  9077   0.0  0.628 Arizona
## 2 0.89  10.3  202  57.9   2.2  5088  25.3  1.555 Boston
## 3 1.43  15.4  113  53.0   3.4  9212   0.0  1.058 Central
## 4 1.02  11.2  168  56.0   0.3  6423  34.3  0.700 Common
## 5 1.49   8.8  192  51.2   1.0  3300  15.6  2.044 Consolid
## 6 1.32  13.5  111  60.0  -2.2 11127  22.5  1.241 Florida
rownames(pubcomp) <- pubcomp[, 9]
pubcomp <- pubcomp[, -9]
```

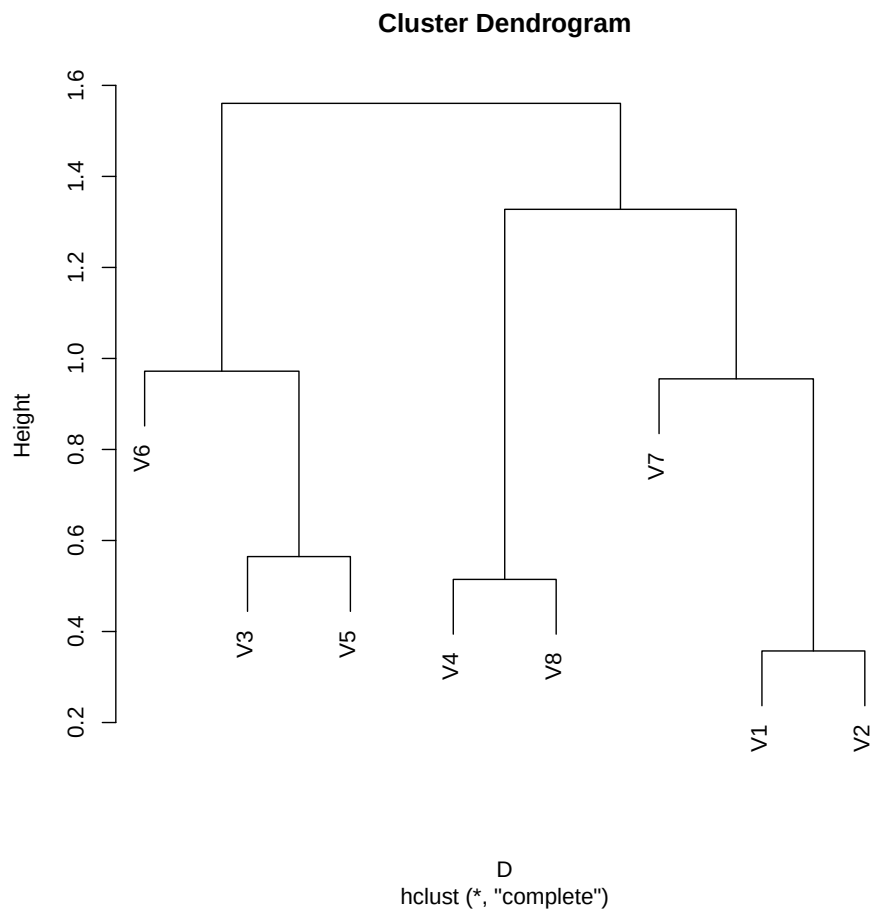
10.34

Example of clustering of variables

```
round(pubcomp.cor <- cor(pubcomp), 3)
##      V1    V2    V3    V4    V5    V6    V7    V8
## V1  1.000  0.643 -0.103 -0.082 -0.259 -0.152  0.045 -0.013
## V2  0.643  1.000 -0.348 -0.086 -0.260 -0.010  0.211 -0.328
## V3 -0.103 -0.348  1.000  0.100  0.435  0.028  0.115  0.005
## V4 -0.082 -0.086  0.100  1.000  0.033 -0.288 -0.164  0.486
## V5 -0.259 -0.260  0.435  0.033  1.000  0.176 -0.019 -0.007
## V6 -0.152 -0.010  0.028 -0.288  0.176  1.000 -0.374 -0.561
## V7  0.045  0.211  0.115 -0.164 -0.019 -0.374  1.000 -0.185
## V8 -0.013 -0.328  0.005  0.486 -0.007 -0.561 -0.185  1.000
D <- 1 - as.dist(pubcomp.cor)
pubcomp.cor.complete <- hclust(D, method = "complete")
```

10.35

```
plot(pubcomp.cor.complete)
```



10.36

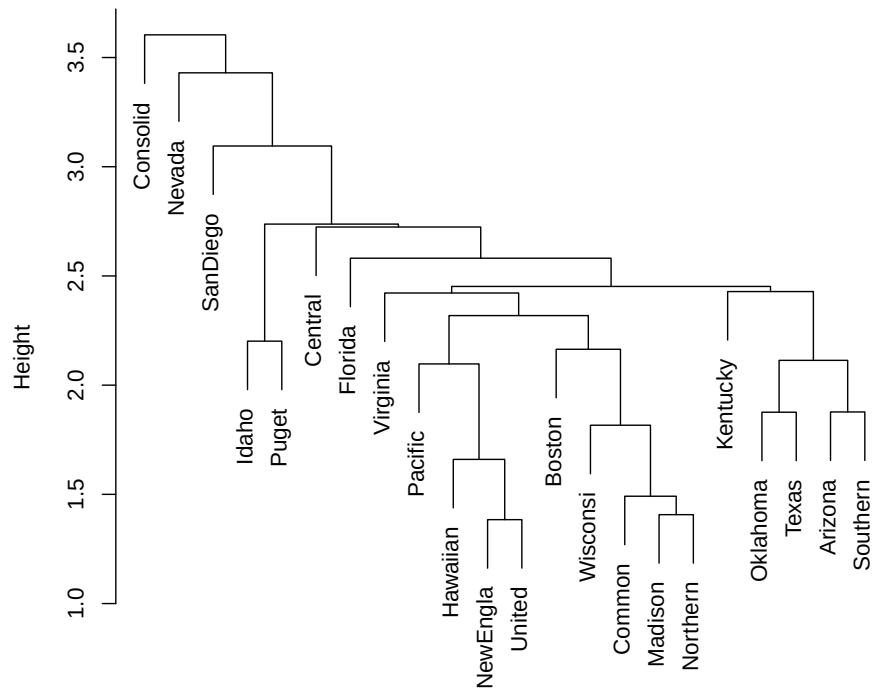
Example of clustering of cases (we consider standardized variables)

```
D <- dist(scale(pubcomp))
```

10.37

```
pubcomp.single <- hclust(D, method = "single")  
plot(pubcomp.single)
```

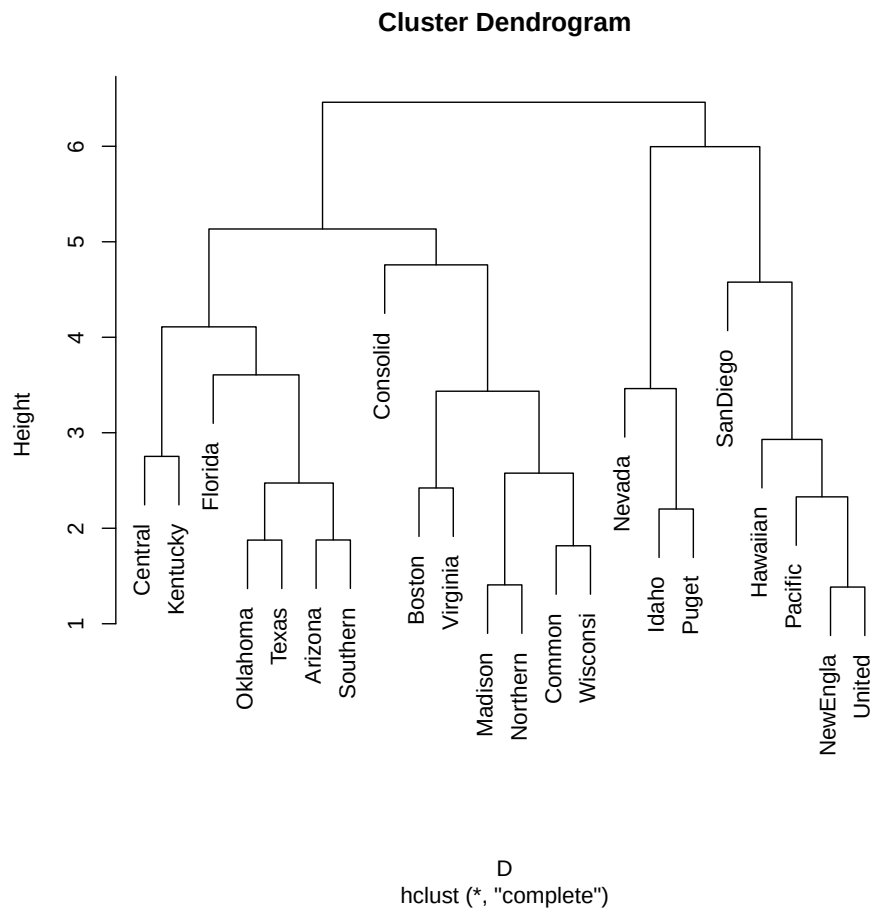
Cluster Dendrogram



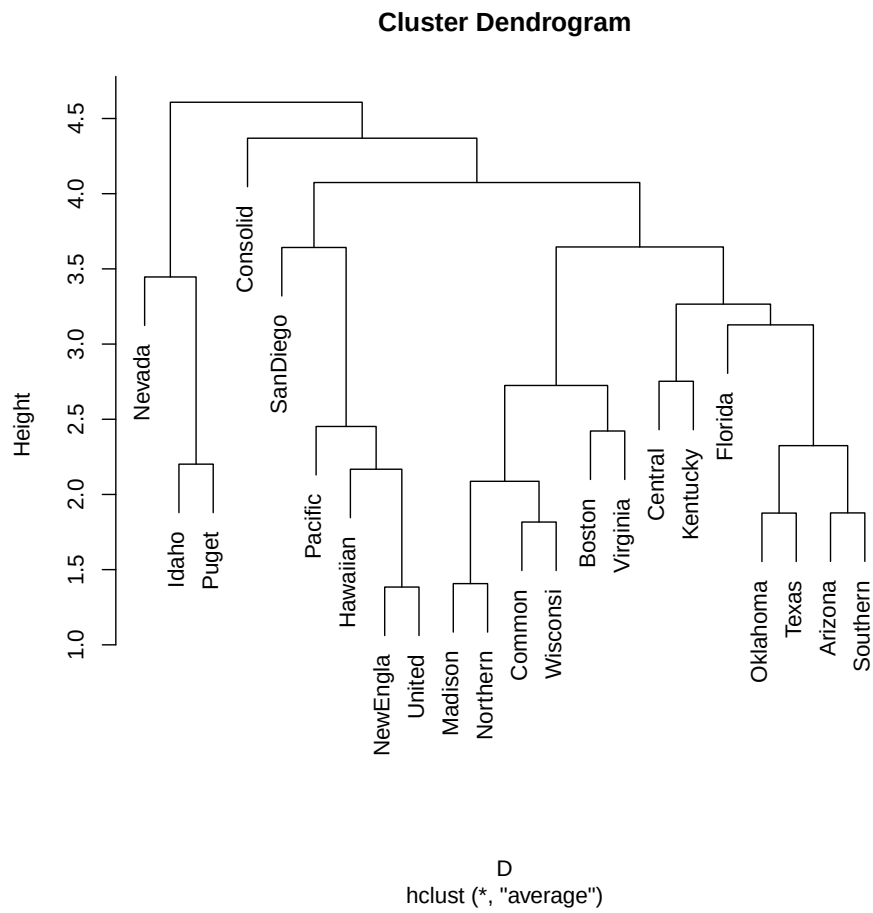
D
hclust (*, "single")

10.38

```
pubcomp.complete <- hclust(D, method = "complete")
plot(pubcomp.complete)
```



```
pubcomp.average <- hclust(D, method = "average")
plot(pubcomp.average)
```



4.2 Classification of Languages

A very simple example of textual considers the classification of languages according to their starting letters of the first 10 numerals.

```
lang <- read.table("numerals.txt", header = TRUE, sep = ",",
  stringsAsFactors = FALSE)
```

10.41

```
lang
##      English Norwegian Danish Dutch German French Spanish
## 1      one          en      en      een      ein      un      uno
## 2      two          to      to      twee     zwei     deux     dos
## 3     three         tre      tre      drie     drei     trois     tres
## 4      four         fire     fire     vier     vier     quatre    cuatro
## 5      five         fern     fem     vijf     funf     cinq     cinco
## 6      six          seks     seks     zes      sechs     six      seix
## 7     seven         sju      syv     zeven     sieben    sept     siete
## 8     eight         atte     otte     acht     acht     huit     ocho
## 9      nine          ni      ni     negen     neun     neuf     nueve
## 10     ten          ti      ti      tien     zehn     dix      diez
##      Italian Polish Hungarian Finnish
## 1      uno      jeden      egy      yksi
## 2      due      dwa      ketto     kaksi
## 3      tre      trzy      harom     kolme
## 4 quattro cztery      negy      neua
## 5 cinque      piec      ot      viisi
## 6      sei      szesc      hat      kuusi
## 7     sette      siedem     het     seiseman
## 8     otto      osiem      nyolc   kahdeksan
## 9     nove      dziewiec   kilenc   ybdeksan
## 10    dieci      dziesiec    tiz     kymmenen
```

10.42

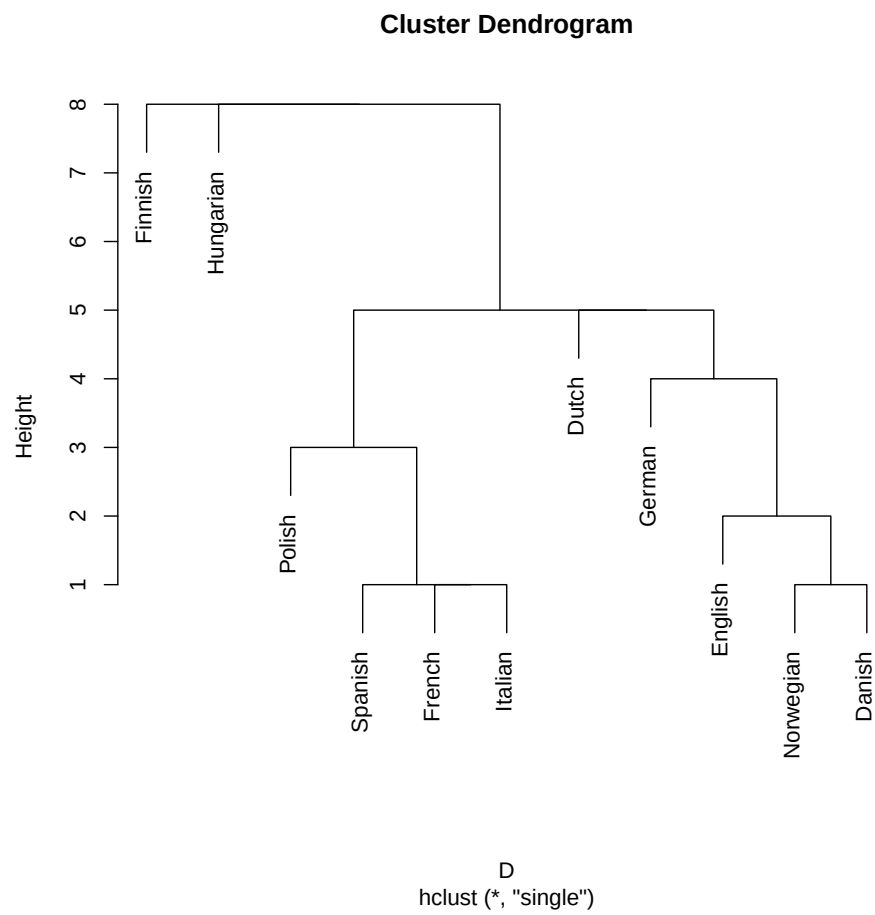
```
lang <- t(as.matrix(lang))
nlang <- nrow(lang)
initials <- substr(as.matrix(lang), 1, 1)
init.conc <- lapply(1:nlang, {
  function(j) sapply(1:nlang, {
    function(i) sum(initials[j, ] == initials[i, ])
  })
})
concordances <- matrix(unlist(init.conc), nlang, nlang)
rownames(concordances) <- colnames(concordances) <- rownames(initials)
```

10.43

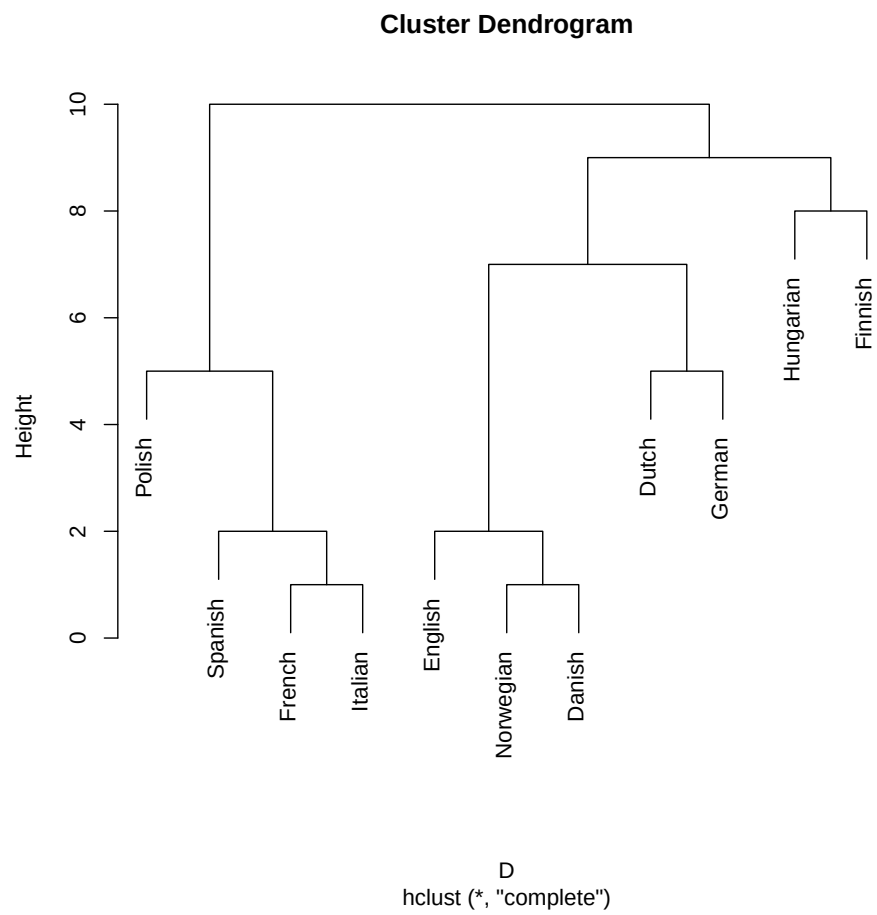
```
10 - concordances
##      English Norwegian Danish Dutch German French
## English      0          2      2      7      6      6
## Norwegian    2          0      1      5      4      6
## Danish       2          1      0      6      5      6
## Dutch        7          5      6      0      5      9
## German       6          4      5      5      0      7
## French       6          6      6      9      7      0
## Spanish      6          6      5      9      7      2
## Italian      6          6      5      9      7      1
## Polish       7          7      6     10      8      5
```

```
## Hungarian      9      8      8      8      9     10
## Finnish        9      9      9      9      9      9
##               Spanish Italian Polish Hungarian Finnish
## English        6      6      7      9      9
## Norwegian      6      6      7      8      9
## Danish         5      5      6      8      9
## Dutch          9      9     10      8      9
## German         7      7      8      9      9
## French         2      1      5     10      9
## Spanish        0      1      3     10      9
## Italian        1      0      4     10      9
## Polish         3      4      0     10      9
## Hungarian     10     10     10      0      8
## Finnish        9      9      9      8      0
D <- as.dist(10 - concordances)
D
##               English Norwegian Danish Dutch German French
## Norwegian      2
## Danish         2      1
## Dutch          7      5      6
## German         6      4      5      5
## French         6      6      6      9      7
## Spanish        6      6      5      9      7      2
## Italian        6      6      5      9      7      1
## Polish         7      7      6     10      8      5
## Hungarian      9      8      8      8      9     10
## Finnish        9      9      9      9      9      9
##               Spanish Italian Polish Hungarian
## Norwegian
## Danish
## Dutch
## German
## French
## Spanish
## Italian        1
## Polish         3      4
## Hungarian     10     10     10
## Finnish        9      9      9      8
```

```
lang.single <- hclust(D, method = "single")
plot(lang.single)
```

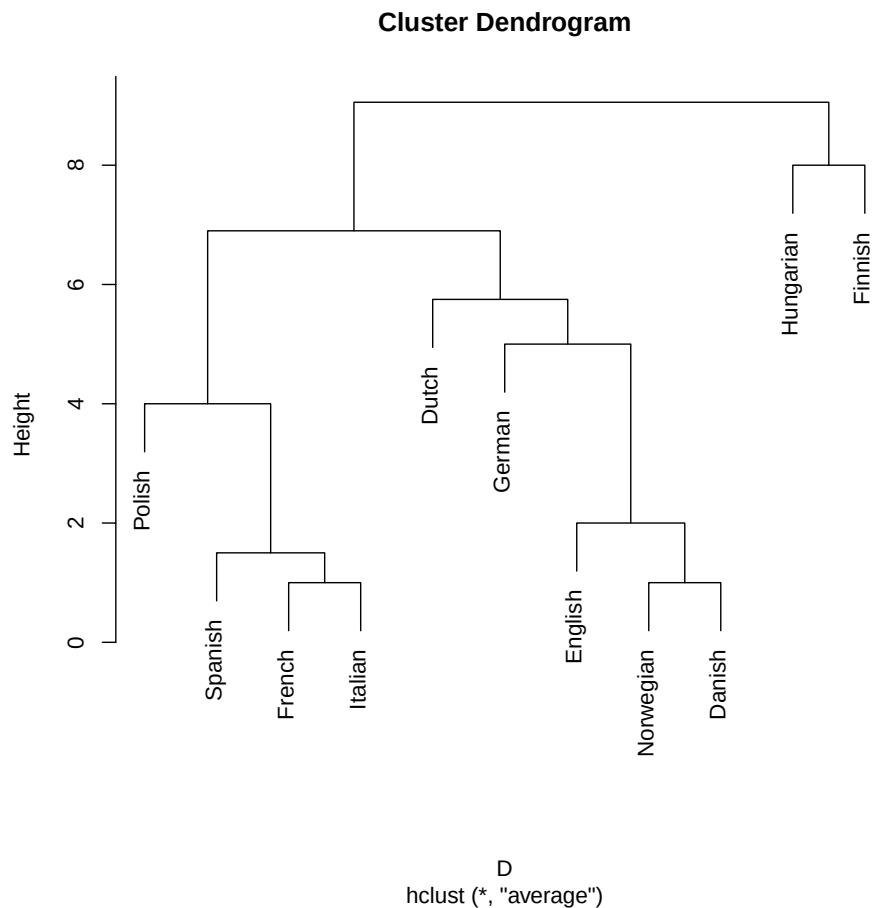



```
lang.complete <- hclust(D, method = "complete")
plot(lang.complete)
```



10.46

```
lang.average <- hclust(D, method = "average")
plot(lang.average)
```



10.47

4.3 Classification of Scotch Whiskies

LaPointe and Legendre (1994) A classification of Pure Malt Scotch Whiskies *Applied Statistics*, 43, 237-257 show motivation and results of the present example.

Geographical reference of results is reported only to show how to build a map. A clustering with spatial contiguity constraint has to be performed to correctly analyze geographical information.

10.48

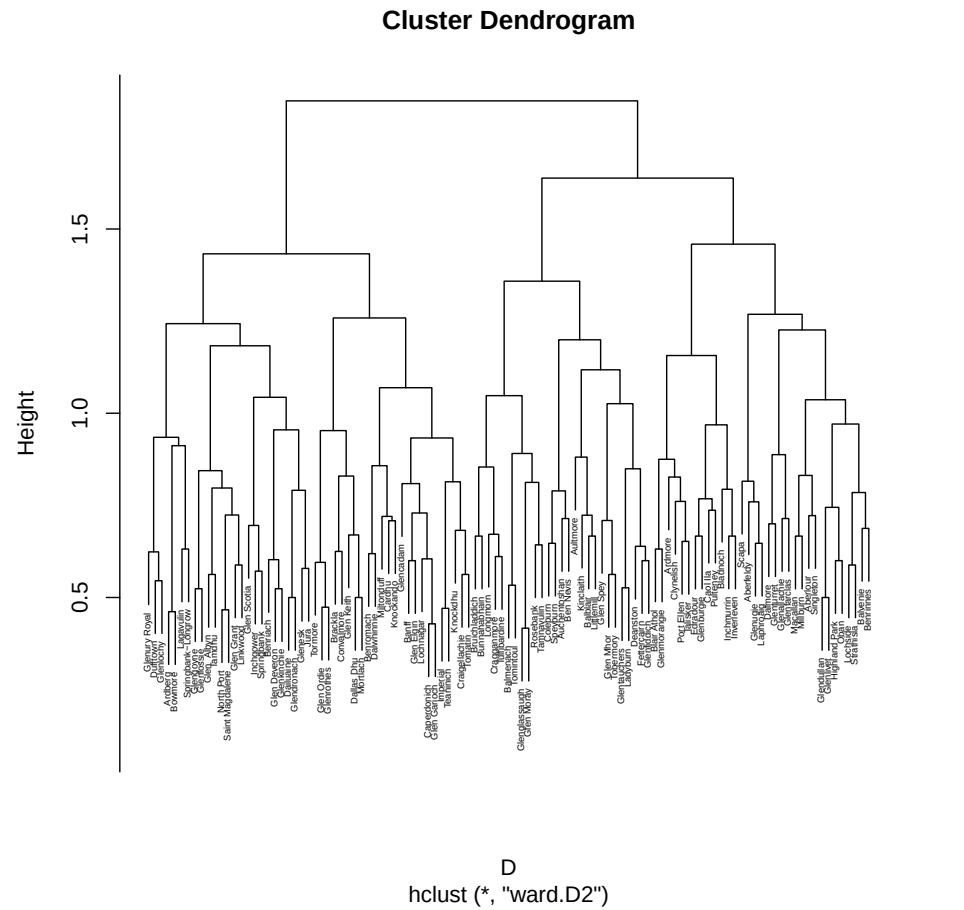
```
data <- read.table("SCOTCH.TXT")
colnames(data) <- c(paste("color", 1:14), paste("nose", 1:12),
  paste("body", 1:8), paste("pal", 1:15), paste("fin", 1:19))
distillery <- read.table("DISTCOOR.TXT", sep = "\t", row.names = 1)
colnames(distillery) <- c("longitude", "latitude")
rownames(data) <- rownames(distillery)

D <- proxy::dist(data, "jaccard")

ave.clus <- hclust(D, method = "ward.D2")
```

10.49

```
plot(ave.clus, cex = 0.4)
```



```
group <- cutree(ave.clus, k = 12)
table(group)
## group
## 1  2  3  4  5  6  7  8  9 10 11 12
## 4 12 7 13 4 12 11 16 10 8  4  8
```

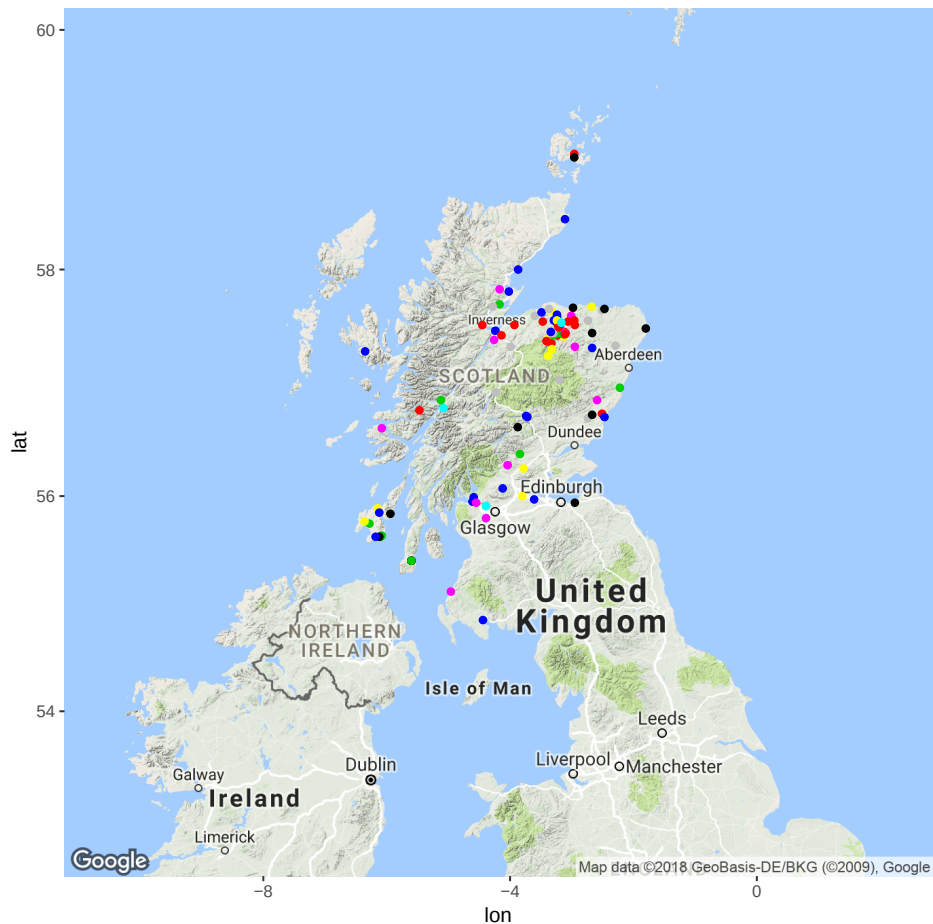
10.50

10.51

The package `ggmap` allows to download maps from Googlemaps¹.

```
library(ggmap)
map <- get_map(location = "scotland", zoom = 6)
mapPoints <- ggmap(map) + geom_point(aes(x = -longitude, y = latitude),
  data = distillery, color = group)
mapPoints
```

10.52



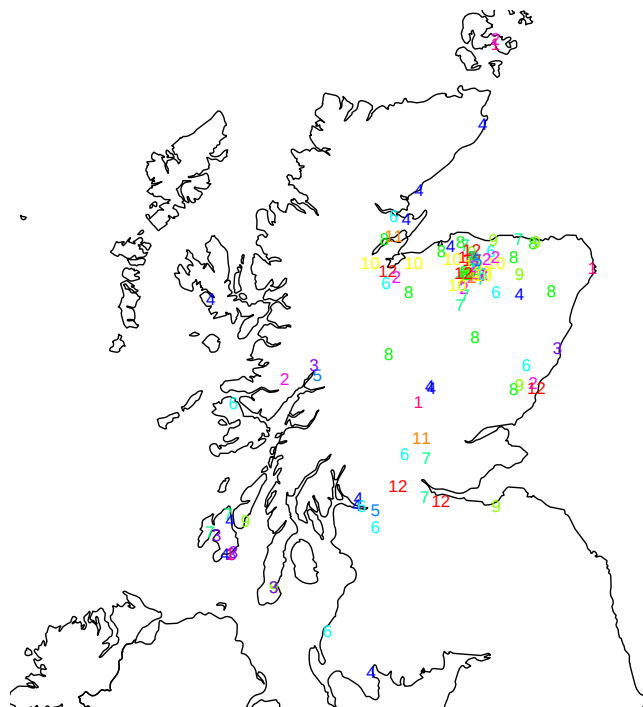
10.53

¹As remarked by David Kahle 'As of mid-2018, the Google Maps Platform requires a registered API key. While this alleviates previous burdens (e.g. query limits), it creates some challenges as well. The most immediate challenge for most R users is that `ggmap` functions that use Google's services no longer function out of the box, since the user has to setup an account with Google, enable the relevant APIs, and then tell R about the user's setup.' Please, read carefully the Details section of the `?ggmap::register_google` help page, before proceeding with setting up a Google account.

If you do not have an account for Google cloud services, the package `rworldmap` contains maps for the whole world.

```
library(rworldmap)
newmap <- getMap(resolution = "high")
plot(newmap, xlim = c(-6.4, -1.8), ylim = c(54.8, 59))
with(distillery, {
  text(-longitude, latitude, group, col = rev(rainbow(12))[group],
    cex = 0.75)
})
```

10.54



10.55